

Product Requirements Document

Autonomous Multi-Hospital Patient Intake, Scheduling & Pre-Visit Voice Agent

Multi-Tenant Healthcare Discovery, Scheduling, Patient Intake & AI Operations Platform

Product Owner: AI.Prof

Version: 2.0 — Concise Candidate Edition

Target: 3–4 Day Prototype

1. Project Overview

Build a **multi-tenant healthcare access and operations platform** that allows multiple hospitals to configure their services, doctors, calendars, availability, questionnaires, and healthcare-system integrations.

Patients interact with the platform through a conversational AI interface, primarily **real-time web voice and telephone**.

Instead of requiring patients to understand hospital departments and booking systems, they should be able to describe their requirement naturally:

"I need to see a doctor for my shoulder pain sometime this week."

The AI should understand the administrative intent, ask for missing information, discover relevant hospitals/doctors, check **real availability**, present suitable options, and execute the booking through controlled application capabilities.

Where an external EHR or healthcare system is configured, the appointment must be created through an integration layer and **verified against the external system before the patient is told that the appointment is confirmed**.

After booking, the platform can execute pre-visit questionnaires, reminders, notifications, synchronization, reconciliation, and other workflows.

The platform must also provide administrators and operators with visibility into AI actions, appointments, integrations, workflows, failures, retries, verification, audit events, and basic AI evaluation.

2. Product Goal

The goal is to demonstrate an **AI-native healthcare access platform**, not simply a chatbot or appointment CRUD application.

The completed system should demonstrate this chain:

None

Patient Request



AI Understanding + Context



Hospital / Doctor Discovery



Real Availability



Authorized Action



Healthcare-System Integration



External Verification



State Synchronization



Workflow / Questionnaire / Notification



Doctor + Admin Visibility

The AI coordinates the process, but it does not make clinical decisions.

3. Core Product Principles

The implementation must follow these principles:

1. **Real availability:** AI must use actual scheduling data rather than inventing slots.
2. **Clarification over guessing:** Ambiguous requests must be clarified.
3. **AI is administrative:** It must not diagnose, prescribe, recommend treatment, or change medication.
4. **Hospital-controlled configuration:** Hospital rules determine what can be booked.
5. **Doctor-controlled availability:** Doctor calendars and blocked periods must be respected.
6. **Capability-based actions:** AI performs actions through controlled interfaces.
7. **External verification:** Do not report success until the external result is verified.
8. **Tenant isolation:** One hospital must never access another hospital's private data.
9. **Useful context:** Retain relevant preferences and appointment context without unnecessarily storing sensitive information.
10. **Observable operations:** Important actions must be traceable.
11. **Recoverable failures:** Integration and workflow failures must have explicit recovery behavior.
12. **Privacy-aware design:** Healthcare information should not unnecessarily appear in logs or AI context.

4. Users & Roles

The platform has four primary user types.

Role	Main Responsibilities
Platform Admin	Approve hospitals, manage global configuration, monitor platform health, AI, integrations, workflows and audit
Hospital Admin	Configure hospital, doctors, calendars, availability, questionnaires, integrations and workflows
Doctor	Manage own availability/calendar and view appointments and authorized pre-visit information
Patient	Register, discover providers, book/reschedule/cancel appointments, complete questionnaires and manage preferences

Tenant boundaries are mandatory.

A hospital administrator can manage their own hospital but cannot access another hospital's private configuration or patient information.

5. Hospital Onboarding

Hospitals register through a self-service process.

Required information may include:

- Hospital/organization name
- Address and contact information
- Departments
- Specialties
- Services
- Operating hours
- Administrator information
- Supported healthcare systems
- Integration configuration

Hospital lifecycle:

None

Draft → Submitted → Under Review → Approved / Rejected

Only approved hospitals can:

- Create active doctors
- Publish availability
- Receive appointments
- Enable production integrations

Platform admins must be able to:

- Review applications
- Approve/reject
- Request corrections
- Suspend/reactivate hospitals
- View hospital activity

6. Hospital & Doctor Configuration

Hospital Configuration

Hospital admins should manage:

- Hospital profile
- Departments
- Specialties
- Doctors
- Appointment types
- Consultation duration
- Calendars
- Working hours
- Availability
- Blocked periods
- Questionnaires
- Communication preferences
- Workflows
- Healthcare-system configuration

Doctor Configuration

Doctor information should include, where applicable:

- Name
- Photo
- Specialty
- Department
- Qualifications
- Experience
- Languages
- Consultation types
- Appointment duration
- Hospital
- External provider ID
- Status

Doctor lifecycle:

Invited → Active → Inactive/Suspended

Only active doctors with valid availability can receive appointments.

7. Scheduling & Availability

Scheduling is a central service and must be the source of truth for bookable slots.

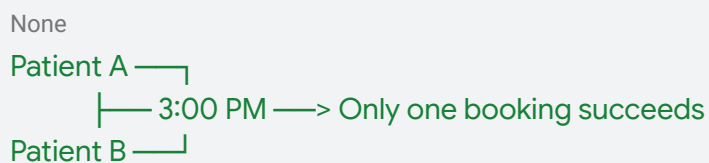
A slot is available only when:

- Doctor is active
- Calendar is active
- Slot is within working hours
- Slot is not blocked
- Doctor is not on leave
- Slot is not already booked
- Appointment type is compatible
- External scheduling constraints are satisfied where applicable

The AI must **never invent availability**.

The system must also prevent concurrent double booking through appropriate transactions, locking, reservation, or conflict-detection mechanisms.

Example:



Availability should be revalidated immediately before booking when required.

8. Patient Management

Patients should be able to:

- Register/login
- Manage profile
- Manage appropriate preferences
- View appointments
- Book appointments
- Reschedule
- Cancel
- Complete questionnaires

Patient data may include:

- Name
- Contact information
- Date of birth
- Communication preference
- Relevant profile information
- Appointment history
- External patient ID where applicable

Follow **data minimization**. Do not store information simply because it is available.

9. AI Patient Access Agent

The AI is the main patient-access interface.

It should support:

- Natural-language understanding
- Administrative intent detection
- Clarification
- Hospital/doctor discovery
- Availability lookup
- Booking
- Rescheduling
- Cancellation
- Appointment lookup

- Questionnaire collection
- Notifications/workflow initiation
- Context retrieval
- External verification
- Human escalation

The AI should be reusable across web voice, telephone, and future channels.

10. AI, Context & Capabilities

These three concerns should remain clearly separated.

Conversation Context

The system may retain:

- Current intent
- Selected hospital
- Selected doctor
- Selected slot
- Current appointment
- Relevant preferences
- Communication preferences
- Completed workflow state

Example:

Patient: "Book Dr. Rao."

AI: "Which of the two available appointments would you prefer?"

If the patient later says:

"Actually, make that Friday."

the AI should resolve "that" using the active conversation state.

If there is insufficient context, it must ask.

Controlled Capabilities

AI actions must go through explicit capabilities such as:

None

search_hospitals
search_doctors
check_availability
lookup_patient
get_appointment
create_appointment
reschedule_appointment
cancel_appointment
get_questionnaire
submit_questionnaire
send_notification
start_workflow
get_context
update_preferences
verify_external_appointment
synchronize_state
transfer_to_human

Every capability should define:

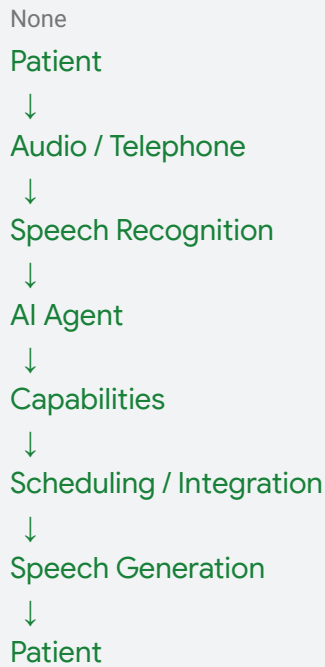
- Input/output schema
- Validation
- Authorization
- Error handling
- Retry behavior
- Idempotency where required
- Verification where required
- Audit information

The AI must **not directly access the database or EHR.**

11. Voice & Telephone Experience

The prototype should support real-time voice interaction.

A conceptual flow is:



The voice experience should support:

- Streaming
- Turn-taking
- Interruption/barge-in
- Silence handling
- Reasonable latency
- Long-running operation handling
- Call failure handling

A target of **sub-2-second perceived latency for normal conversational turns** is desirable where technically achievable.

Telephone functionality should support:

- Inbound calls
- Patient identification

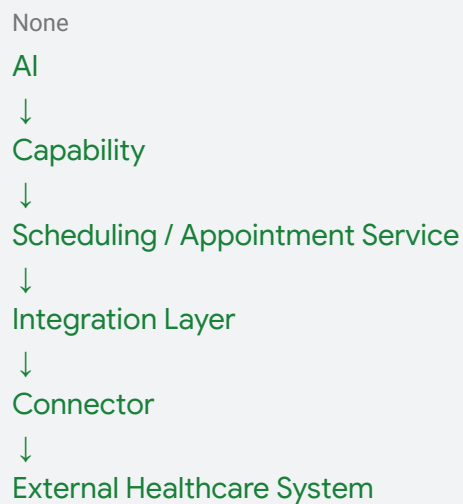
- Appointment lookup
- Booking
- Rescheduling
- Cancellation
- Failure handling
- Human escalation

12. Healthcare-System / EHR Integration

The prototype should include a **Mock EHR or Mock Healthcare System**.

The architecture must allow a real connector to replace the mock later.

The integration structure should be:



The integration layer must hide vendor-specific implementation details.

Required operations:

- Patient lookup
- Provider lookup
- Facility/department lookup
- Calendar lookup where applicable
- Availability lookup where supported
- Appointment creation
- Appointment update

- Appointment cancellation
- Appointment rescheduling
- Appointment retrieval
- Appointment verification

Maintain mappings such as:

None

Internal Patient ↔ External Patient

Internal Doctor ↔ External Provider

Internal Appointment ↔ External Appointment

Internal Facility ↔ External Facility

13. Verification, Failure Recovery & Reconciliation

This is a critical part of the challenge.

A successful API response does not automatically mean the external operation succeeded.

For bookings:

None

Create Appointment



External Response



Verify External Record



Synchronize Internal State



Confirm to Patient

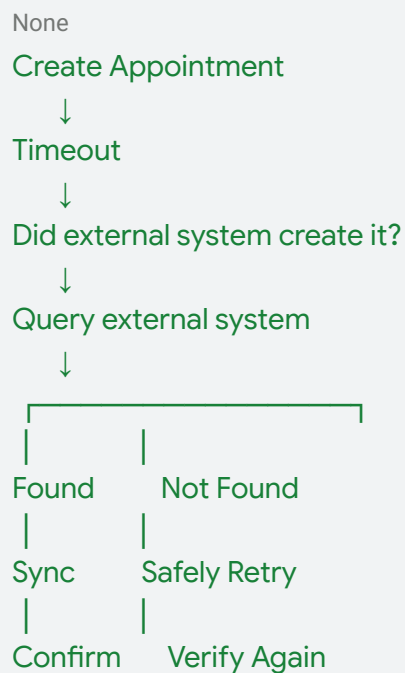
The platform must handle:

- Timeout

- Network failure
- Authentication failure
- Authorization failure
- Rate limiting
- External-system outage
- Validation errors
- Mapping errors
- Missing patient/provider
- Slot conflict
- Duplicate request
- Partial success
- Unknown outcome

Unknown Outcome

For example:



The system must not blindly retry operations that could create duplicate appointments.

Unresolved cases should become **reconciliation records** or human escalations.

14. Appointment Management

Appointment operations:

- Create
- Confirm
- Reschedule
- Cancel
- Retrieve
- Notify
- Synchronize
- Reconcile

Suggested states:

None

Requested

Pending

Confirmed

Rescheduled

Cancelled

Completed

No-show

Failed

Synchronization Pending

Reconciliation Required

Each appointment should maintain appropriate history and internal/external identifiers.

Booking

The patient selects a real available slot. The platform creates the appointment through the controlled booking capability, performs external integration where applicable, verifies the result, and only then confirms success.

Rescheduling

The platform should identify the existing appointment, find a valid new slot, update the appointment, verify the external result, synchronize state, and release the old slot.

Cancellation

The platform should identify the correct appointment, execute cancellation, verify the external result where applicable, synchronize state, release availability, and notify relevant parties.

15. Pre-Visit Questionnaire

Hospital/doctor administrators can configure approved questionnaires.

Questionnaires can contain:

- Yes/no
- Choice
- Multiple choice
- Numeric
- Date
- Short text
- Long text
- Structured fields

Questionnaires may be associated with:

- Hospital
- Specialty
- Doctor
- Appointment type
- Approved condition/category

The AI can collect answers conversationally and store structured responses.

Example:

None

Appointment Booked



Questionnaire Assigned



AI Collects Responses



Structured Responses

↓
Doctor Reviews

The AI must not invent diagnostic or treatment questions.

The questionnaire system must not be used for:

- Diagnosis
- Prescribing
- Medication changes
- Independent clinical assessment

Potentially urgent information should follow a predefined escalation policy.

16. Workflow & Event Automation

The system should support asynchronous and scheduled workflows.

Important events may include:

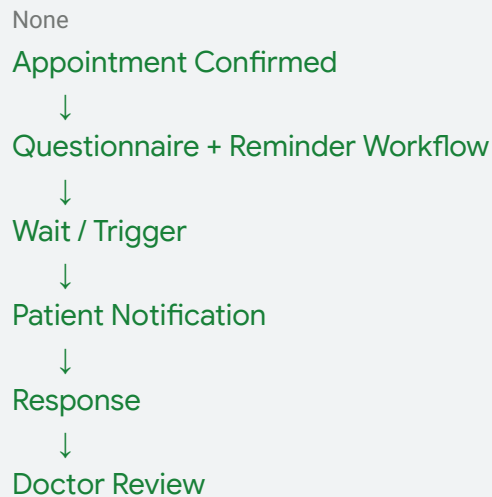
- Hospital approved
- Doctor created
- Appointment booked
- Appointment cancelled
- Appointment rescheduled
- Questionnaire assigned
- Questionnaire completed
- AI conversation started
- Capability executed
- EHR operation started/completed/failed
- External verification completed
- Reconciliation required
- Workflow started/completed/failed
- Human escalation

These events can trigger:

- Notifications
- Reminders

- Questionnaires
- Synchronization
- Recovery
- Analytics
- Audit
- Evaluation

Example:



Workflows should support:

- Delays
- Conditions
- Retries
- Idempotency
- Execution history
- Failure states

17. Notifications

The platform should support configurable notifications.

Patient

- Appointment confirmation
- Reminder

- Cancellation
- Rescheduling
- Questionnaire reminder
- Important updates

Doctor

- New appointment
- Cancellation
- Rescheduling
- Questionnaire completion
- Upcoming appointment

Hospital

- Application status
- Appointment activity
- Integration failures
- Operational alerts

18. Dashboards

The platform should provide role-specific dashboards with enough functionality to demonstrate the complete system without turning the project into a large frontend exercise. The **Platform Admin** dashboard should provide an overview of hospital applications, hospitals, doctors, patients, appointments, AI activity, integration activity, workflows, analytics, AI evaluation, operational health, and audit logs. The **Hospital Admin** dashboard should provide access to hospital overview, appointments, doctors, calendars, availability, questionnaires, AI activity, integration activity, workflows, analytics, integrations, and staff/access management. The **Doctor** dashboard should provide today's and upcoming appointments, calendar, availability, blocked time, appointment details, questionnaires, and authorized pre-visit responses. The **Patient** dashboard should provide a home view, AI assistant, upcoming and historical appointments, questionnaires, preferences, and profile management.

19. Analytics, Observability & Audit

The platform must make important operations traceable.

A booking should be traceable across:

None

Conversation

→ AI decision

→ Capability

→ Scheduling

→ EHR operation

→ Verification

→ Synchronization

→ Workflow

→ Notification

Use a correlation/operation ID to connect related events.

Metrics

Track useful metrics such as:

AI

- Conversations
- Latency
- Capability success/failure
- Escalation
- Usage
- Approximate cost

Scheduling

- Booking success
- Availability
- Cancellations
- Rescheduling
- Utilization

Integration

- Requests
- Success/failure
- Verification
- Retry
- Recovery
- Reconciliation
- Unknown outcomes

Workflow

- Running
- Completed
- Failed
- Retried
- Duration

Audit

Record important events such as:

- Login/access
- Appointment operations
- Patient-data access
- AI actions
- Capability executions
- Integration operations
- Configuration changes
- Administrative actions

Avoid unnecessarily storing raw sensitive healthcare content in operational logs.

20. AI Safety & Healthcare Boundaries

The AI is an **administrative access assistant**.

It may

- Find hospitals
- Find doctors
- Find appointments
- Book appointments
- Reschedule/cancel
- Answer approved administrative questions
- Ask approved pre-visit questions
- Record patient responses
- Trigger workflows
- Communicate appointment status
- Escalate to humans

It must not

- Diagnose
- Prescribe
- Change medication
- Recommend treatment
- Make independent clinical assessments
- Invent clinical information

The AI should distinguish:

"You reported chest discomfort."

from:

"You have a heart condition."

The first reflects patient input. The second is an unsupported clinical conclusion.

21. Security & Privacy

The prototype must demonstrate:

- Authentication
- Authorization
- Role-based access control
- Tenant isolation
- Resource ownership
- Secure API design
- Input validation
- Secure secret management
- Audit logging
- Appropriate encryption
- Privacy-aware logging

Critical rule:

Hospital A must never be able to access Hospital B's private data.

The same restriction applies to:

- Patient context
- Conversations
- Questionnaires
- Appointments
- Integration data
- AI activity

External credentials, API keys, database credentials, voice credentials, and AI provider secrets must never be hardcoded.

22. Data Model

The implementation should use explicit entities for the major platform concepts, including Platform, Hospital, Hospital Admin/Staff, Department, Specialty, Doctor, Calendar, Availability, Blocked Slot, Patient, User Context/Preferences, Appointment, Questionnaire, Questionnaire Response, AI Conversation, AI Context, Capability, Capability Execution, Healthcare-System Connection, External Identifier Mapping, Integration Operation, Integration Verification, Reconciliation Record, Workflow, Workflow Execution, Notification, AI Evaluation, Audit Event, and Operational Event. The data model should clearly define ownership and

relationships while supporting tenant isolation, external identifiers, state history, efficient lookups, and appropriate indexing.

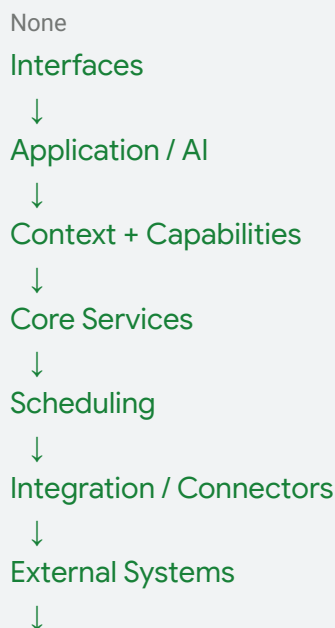
23. State Management

Different types of application state must be kept separate rather than being combined into a single unstructured state object. Transactional state should represent durable business state such as an appointment being Confirmed; conversational state should represent the current interaction context such as the selected doctor; user context should store preferences such as a preferred appointment time; workflow state should track process progress such as a scheduled reminder; integration state should represent external-system status such as an EHR appointment being Verified; and operational state should capture system conditions such as a reconciliation being Required.

The platform must not store the entire application state as an unstructured AI memory object. Business, conversational, workflow, integration, and operational states should remain explicitly represented so that they can be validated, queried, updated, audited, recovered, and synchronized independently.

24. Architecture Requirements

The implementation should separate the following logical components:



Verification / Synchronization



Events / Workflows



Data / Analytics / Observability

The candidate is free to choose the actual technology and deployment architecture.

However:

- AI logic should not contain vendor-specific EHR logic.
- Scheduling should remain independent from conversation logic.
- External integrations should be behind connectors/interfaces.
- Long-running workflows should be asynchronous.
- Operational events should be separate from transactional records.
- AI actions should be structured and auditable.

25. Reliability & Idempotency

The platform should protect against duplicate actions.

Especially:

- Appointment creation
- Appointment cancellation
- Rescheduling
- Notifications
- Workflow execution
- External integration requests

Use appropriate:

- Idempotency keys
- Correlation IDs
- Transactions
- Conflict detection
- External verification

The candidate should demonstrate at least one scenario where a failure occurs after an external request and the system safely determines the actual final state.

26. Testing

Testing should focus on core business behavior.

Unit Tests

At minimum:

- Availability calculation
- Slot validation
- Appointment state transitions
- Context resolution
- Capability validation
- Workflow conditions
- Identifier mapping
- Idempotency
- Reconciliation

Integration Tests

At minimum:

- AI → scheduling
- Scheduling → appointment
- Appointment → EHR
- EHR → verification
- Verification → synchronization
- Booking → workflow
- Workflow → notification

AI Tests

Test:

- Intent
- Context

- Clarification
- Tool/capability selection
- Invalid requests
- Unsupported requests
- Safety boundaries

EHR Tests

Test:

- Patient mapping
- Provider mapping
- Appointment creation
- Rescheduling
- Cancellation
- Timeout
- Duplicate requests
- Unknown outcomes
- Verification
- Reconciliation

End-to-End

The minimum successful path is:

None

Patient

→ Voice

→ AI

→ Discovery

→ Availability

→ Booking

→ Mock EHR

→ Verification

→ Synchronization

→ Questionnaire

→ Workflow

→ Doctor View

→ Admin Analytics

Also demonstrate **at least one failure/recovery path**.

27. Definition of Done

The project is considered complete when the following scenario works end-to-end:

Hospital

Hospital registers → admin approves → hospital is configured → doctor is created → calendar and availability are configured → healthcare-system integration is configured.

Patient

Patient registers → starts voice conversation → describe requirements → AI understands intent → resolves context → finds relevant doctors/hospitals → checks actual availability → patient selects slot.

Booking

Appointment is created → external healthcare system is called → external result is verified → internal state is synchronized → patient receives confirmation.

Follow-Up

Questionnaire is assigned → patient answers conversationally → responses are structured → doctor can review → reminder/notification workflows execute.

Operations

Admin can see relevant appointments, AI, integration, workflow, audit, and basic operational metrics.

Failure

At least one meaningful external or workflow failure is demonstrated and recovered safely.

28. Required Failure Demonstration

Candidates must demonstrate one scenario such as:

Option A — EHR Failure

None

Booking



EHR Timeout



Failure Classification



Retry



External Verification



Recovered



Synchronize



Confirm

Option B — Unknown Outcome

None

Booking Sent



Network Timeout



Unknown Result



Query EHR



Appointment Found



Synchronize



Do NOT Create Duplicate

Option C — Unrecoverable Failure

None
EHR Failure
↓
Retries Exhausted
↓
External State Check
↓
Reconciliation Record
↓
Human Escalation
↓
Operational Dashboard

This is an important part of the evaluation.

29. Submission Requirements

Submit:

1. Deployed Application

A publicly accessible deployment demonstrating the main workflow.

2. GitHub Repository

Include:

- Source code
- Setup instructions
- Environment template
- Database setup
- Tests
- Architecture documentation
- AI documentation
- Integration documentation
- Deployment instructions

No secrets in the repository.

3. Demo Video

Show the **connected workflow**, not only individual screens:

None

Hospital Setup

→ Doctor

→ Availability

→ Patient

→ AI Voice

→ Booking

→ EHR

→ Verification

→ Questionnaire

→ Workflow

→ Doctor

→ Admin

Include the failure/recovery scenario.

4. Architecture Documentation

Include:

- High-level architecture
- Data model
- Booking sequence
- EHR integration flow
- Failure/recovery flow
- Security/tenant model

5. AI Documentation

Document:

- AI models
- Voice technology
- AI development tools
- Runtime AI
- Important prompts
- Evaluation approach

6. README

Include:

- Overview
- Features
- Architecture
- Tech stack
- Setup
- Environment variables
- AI setup
- Voice setup
- Mock EHR
- Workflows
- Tests
- Deployment
- Credentials/demo accounts
- Known limitations
- Future improvements

30. Technology Selection

No specific stack is required.

Candidates should justify the major technology choices based on:

- Prototype speed
- Reliability
- Maintainability
- AI support
- Real-time capabilities
- Integration support
- Data modeling
- Observability
- Cost
- Deployment simplicity

The evaluation focuses on **engineering decisions**, not framework preference.

31. Evaluation Focus

The project will primarily be evaluated on:

Product Completion

Does the main patient journey actually work?

AI Quality

Can the AI understand intent, maintain context, clarify ambiguity, and execute appropriate capabilities?

Engineering

Is the application structured cleanly and maintainably?

Scheduling

Does it use real availability and prevent double booking?

Integration

Is the EHR layer properly separated and verified?

Reliability

Can the system safely recover from failures?

Security

Are tenant boundaries and permissions enforced?

Observability

Can an operator understand what happened during a booking?

UX

Does the patient experience feel conversational and natural?

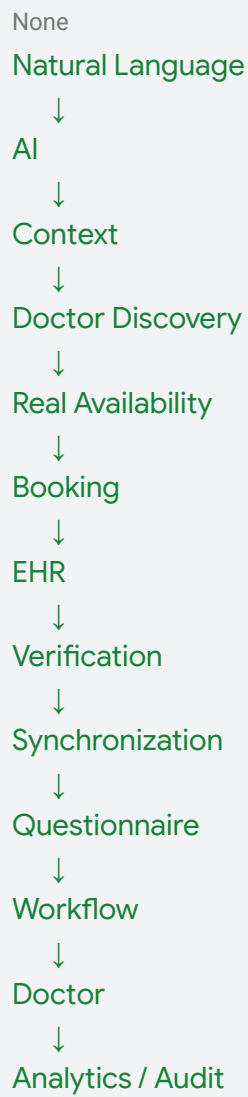
Scope Management

Did the candidate prioritize the core workflow instead of building many disconnected features?

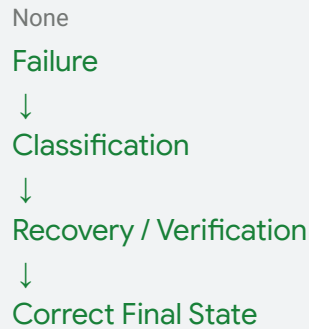
32. Final Success Criteria

The strongest implementation will feel like a **small but coherent healthcare platform**, not a collection of demos.

The evaluator should be able to follow one patient request from:



and then intentionally cause a failure to observe:



The central principle is:

The AI should not merely talk about performing healthcare operations. It should safely coordinate real application actions, integrate with external systems, verify outcomes, recover from failures, and leave the platform in a correct and observable state.

33. Final Product Statement

The final prototype should demonstrate:

A multi-tenant, AI-native healthcare access platform where hospitals configure services, doctors control availability, patients communicate naturally, AI understands and coordinates requests, capabilities execute authorized actions, scheduling provides real availability, healthcare-system integrations perform external operations, external results are verified, internal state is synchronized, workflows automate follow-up, and operators can observe and evaluate the complete process.

The priority is depth of one complete reliable workflow over breadth of disconnected features.